

МЕТОДИЧКА

Обучение с подкреплением: задачи, в которых требуется оценка функции

Подробный учебный материал, пример на Python, разбор кода и практические задания

Содержание документа:

1. Что означает «оценка функции» в обучении с подкреплением
2. Типы задач RL, где оценка функции является центральной частью решения
3. Характеристики состояния системы и требования к описанию состояния
4. Алгоритмы машинного обучения для оценки функции ценности
5. Подробный пример на Python: оценка $V(s)$ методом TD(0) в GridWorld
6. Интерпретация результатов и типичные ошибки
7. Двадцать заданий
8. Контрольные вопросы и рекомендации к отчету

1. Что означает «оценка функции» в обучении с подкреплением

В обучении с подкреплением агент взаимодействует со средой, наблюдает состояние, выбирает действие, получает награду и переходит в новое состояние. Главная трудность заключается в том, что агенту заранее неизвестно, какие действия приведут к наибольшей суммарной награде в долгосрочной перспективе. Поэтому ему приходится оценивать не только мгновенную выгоду, но и ожидаемую полезность будущих состояний и действий.

Под оценкой функции в задачах RL обычно понимают построение приближенной зависимости, которая по описанию состояния или пары «состояние–действие» возвращает численную оценку ожидаемой долгосрочной выгоды. Чаще всего оцениваются функция ценности состояния $V(s)$, функция ценности действия $Q(s, a)$ и функция преимущества $A(s, a)$.

Если пространство состояний мало, можно хранить отдельное число для каждого состояния в таблице. Однако в реальных задачах число состояний может быть очень большим или даже бесконечным. Тогда приходится строить модель, которая умеет обобщать и по похожим признакам состояния выдавать разумную оценку даже для тех состояний, которые раньше встречались редко или не встречались вовсе.

Таким образом, задачи с оценкой функции находятся на стыке RL и машинного обучения. С одной стороны, агент учится по взаимодействию со средой. С другой стороны, он использует методы регрессии, аппроксимации и оптимизации для подстройки функции ценности.

Объект оценки	Обозначение	Смысл
Ценность состояния	$V(s)$	Ожидаемая суммарная награда при нахождении в состоянии s и дальнейшем следовании стратегии
Ценность действия	$Q(s, a)$	Ожидаемая суммарная награда, если в состоянии s выполнить действие a и далее следовать стратегии
Преимущество	$A(s, a)$	Насколько действие a лучше или хуже среднего поведения в состоянии s

2. Типы задач RL, где оценка функции является центральной частью решения

Оценка функции особенно важна в тех задачах, где недостаточно просто подобрать действие по текущей награде. Ниже приведены типичные классы задач, в которых оценка функции является обязательным элементом.

2.1. Оценка состояния при заданной стратегии

Здесь стратегия уже задана или фиксируется на некотором шаге алгоритма. Требуется оценить, насколько хороши разные состояния при следовании этой стратегии. Такая постановка встречается в динамическом программировании, методах Монте-Карло, TD-обучении и в шаге оценки стратегии алгоритма policy iteration.

Практический смысл: если мы можем оценить $V(s)$, мы понимаем, в какие состояния желательно попадать и какие состояния опасны или бесперспективны.

2.2. Оценка действий для выбора наилучшего действия

Во многих прикладных задачах нужно оценивать не просто состояние, а именно пару «состояние–действие». Тогда агент может сравнивать разные действия в одном и том же состоянии и выбирать действие с наибольшей оценкой $Q(s, a)$.

Так работают Q-learning, SARSA, Expected SARSA и многие современные модификации, где основная трудность переносится на точную оценку функции Q .

2.3. Большие и непрерывные пространства состояний

Если состояние описывается десятками или сотнями параметров, табличный подход становится невозможным. Например, это управление роботом, регулирование производственного процесса, логистика, энергосистемы, торговые и рекомендательные системы. Здесь оценка функции строится как аппроксимация по признакам.

2.4. Частично наблюдаемые и шумные среды

Если наблюдение не совпадает с истинным состоянием среды, агенту приходится работать с признаковым представлением, агрегированными характеристиками истории или скрытым

состоянием модели. Оценка функции должна быть устойчивой к шуму и неполноте информации.

2.5. Задачи с редкими наградами

Когда награда появляется редко, по одному эпизоду трудно понять, что было полезным. Функция ценности позволяет распространять информацию о будущем вознаграждении назад по цепочке переходов и тем самым ускоряет обучение.

3. Характеристики состояния системы и требования к описанию состояния

Состояние — это информационное описание текущей ситуации, которого должно быть достаточно для принятия решения. В идеальном случае состояние обладает свойством Маркова: будущее зависит от прошлого только через текущее состояние. На практике это свойство достигается не всегда, поэтому признаки состояния приходится подбирать carefully и предметно.

Качество оценки функции напрямую зависит от того, насколько хорошо выбраны характеристики состояния. Если состояние описано бедно, функция ценности не сможет различать действительно разные ситуации. Если описание содержит слишком много лишнего, обучение станет медленным и нестабильным.

Характеристика	Что означает	Практическое замечание
Полнота	Состояние должно содержать существенную для решения информацию	Иначе нарушается свойство Маркова и ценность становится смазанной
Компактность	Нужно избегать лишних признаков	Чрезмерная размерность ухудшает обучение и увеличивает требования к данным
Информативность	Признаки должны различать выгодные и невыгодные ситуации	Полезны расстояния до цели, остаток ресурса, загрузка системы, временные признаки
Нормируемость	Желательно приводить признаки к сопоставимым масштабам	Это особенно важно для линейных моделей и нейросетей
Устойчивость к шуму	Признаки не должны резко менять смысл из-за небольших колебаний	Стоит использовать сглаживание, агрегаты и робастные преобразования

Типичные компоненты состояния в прикладных задачах:

- текущие физические параметры системы: координаты, скорость, температура, запас ресурсов;
- контекст выполнения: номер шага, время суток, этап процесса, активный режим;
- история или агрегаты: среднее за последние шаги, накопленные потери, скользящие характеристики;
- ограничения: остаток бюджета, лимиты, доступность ресурсов, состояние очередей;

- признаки риска: вероятность отказа, близость к запрещенной области, перегрузка, штрафные состояния.

4. Алгоритмы машинного обучения для оценки функции ценности

Ниже собраны основные алгоритмические подходы. Одни оценивают функцию ценности в явном табличном виде, другие строят параметрическую модель по признакам. Выбор подхода зависит от размера пространства состояний, наличия непрерывных признаков, количества данных и требований к интерпретируемости.

Подход	Идея	Когда уместен
Табличные методы	Для каждого состояния или пары «состояние–действие» хранится отдельная оценка. Обновления производятся напрямую по формуле Монте-Карло, TD(0), SARSA, Q-learning.	Хороши в маленьких задачах, дают прозрачную интерпретацию, но не масштабируются.
Линейная аппроксимация	Функция ценности выражается как взвешенная сумма признаков: $V(s) \approx w^T x(s)$. Параметры обновляются градиентно.	Быстро обучается, хорошо объяснима, требует качественного признакового описания.
Полиномиальные и базисные функции	Признаки строятся не напрямую, а через нелинейные преобразования: полиномы, радиальные базисные функции, тайловое кодирование.	Позволяют аппроксимировать нелинейные зависимости при умеренной сложности модели.
Деревья решений и ансамбли	Оценка строится методами регрессии: decision trees, random forest, gradient boosting.	Могут применяться как вспомогательные аппроксиматоры по накопленному буферу опыта, но в RL требуют аккуратного обновления из-за нестационарности данных.
Нейронные сети	Функция ценности аппроксимируется глубокой сетью; вход — признаки состояния, выход — $V(s)$ или вектор $Q(s, a)$.	Подход пригоден для сложных и больших задач, но требует настройки, нормализации, буфера опыта и регуляризации.
Actor–Critic и критик	Критик — это модель, оценивающая $V(s)$ или $Q(s, a)$, а актер улучшает стратегию на основе этих оценок.	Оценка функции становится внутренней частью совместного процесса обучения стратегии.

4.1. Монте-Карло

Методы Монте-Карло оценивают функцию ценности по полным эпизодам. После завершения эпизода для каждого посещенного состояния вычисляется полный возврат G_t , и функция подстраивается под это значение. Плюс метода — отсутствие смещения при корректных предположениях. Минус — высокая дисперсия и необходимость дожидаться окончания эпизода.

4.2. TD-обучение

Temporal Difference обновляет оценку сразу после перехода, используя идею бутстрэппинга: текущая оценка подправляется в сторону $r + \gamma V(s')$. Это делает обучение более оперативным и пригодным для длинных или непрерывных эпизодов.

4.3. Q-learning и SARSA

Эти алгоритмы оценивают уже не $V(s)$, а $Q(s, a)$. Q-learning учится по жадной оценке следующего шага, а SARSA — по реально выбранному действию. Оба алгоритма можно сочетать с аппроксимацией функции.

4.4. Линейные методы и градиентные обновления

Когда табличная оценка невозможна, параметры модели обновляются по градиенту ошибки. Например, для линейной модели веса корректируются пропорционально TD-ошибке и вектору признаков.

4.5. Глубокое обучение

Если состояние — это изображение, длинный вектор датчиков или сложный многомерный контекст, используются нейросети. Наиболее известен DQN, где сеть оценивает $Q(s, a)$, а обучение стабилизируется реплей-буфером и целевой сетью.

5. Подробный пример на Python: оценка $V(s)$ методом TD(0) в GridWorld

Ниже рассматривается учебная задача, в которой агент не ищет новую стратегию, а оценивает ценность состояний при фиксированной стратегии поведения. Это очень полезная постановка: она позволяет изолированно понять, как происходит оценка функции $V(s)$.

Среда представляет собой поле 4×4 . Из большинства клеток можно перемещаться вверх, вниз, влево и вправо. Есть терминальное состояние «цель», за попадание в которое агент получает положительную награду. За обычный шаг начисляется небольшой отрицательный штраф, чтобы поощрять короткие пути. Политика поведения выбирает действия почти случайно, но с небольшим смещением в сторону цели.

Задача алгоритма — по многократным эпизодам вычислить приближенные значения $V(s)$, показывающие, насколько перспективна каждая клетка поля.

Полный код учебного примера приведен ниже. В методичке он дан с комментариями не для компактности, а для пошагового понимания логики.

```
import random

# =====
# Учебный пример: оценка функции ценности состояния  $V(s)$ 
```

```

# методом TD(0) в небольшой дискретной среде GridWorld.
#
# Идея примера:
# 1. У нас есть среда из клеток.
# 2. Агент действует по фиксированной стратегии.
# 3. Мы НЕ улучшаем стратегию, а только оцениваем,
#    насколько полезно каждое состояние.
# 4. Для оценки используем TD(0):
#    
$$V(s) \leftarrow V(s) + \alpha * (r + \gamma * V(s') - V(s))$$

# =====

```

```

class GridWorld:
    def __init__(self, width=4, height=4, goal=(3, 3)):
        self.width = width
        self.height = height
        self.goal = goal
        self.start = (0, 0)

    def reset(self):
        # Возвращаем стартовое состояние эпизода
        return self.start

    def is_terminal(self, state):
        # Терминальное состояние: цель
        return state == self.goal

    def step(self, state, action):
        # Делаем переход по действию.
        # Если выходим за границу поля, остаемся на месте.
        x, y = state

        if action == "up":
            y = max(0, y - 1)
        elif action == "down":
            y = min(self.height - 1, y + 1)
        elif action == "left":
            x = max(0, x - 1)
        elif action == "right":
            x = min(self.width - 1, x + 1)

        next_state = (x, y)

        # Награда:
        # - небольшой штраф за каждый обычный шаг;
        # - положительная награда за достижение цели.
        if next_state == self.goal:
            reward = 10.0
        else:
            reward = -0.2

        done = self.is_terminal(next_state)

```

```
    return next_state, reward, done
```

```
def policy(state, env):  
    # Фиксированная стратегия поведения.  
    # Мы не выбираем оптимальное действие,  
    # а просто задаем вероятностное правило.  
    # Небольшое смещение в сторону цели помогает  
    # быстрее собрать осмысленные данные.  
    actions = ["up", "down", "left", "right"]  
    x, y = state  
    goal_x, goal_y = env.goal  
  
    preferred = []  
    if x < goal_x:  
        preferred.append("right")  
    if y < goal_y:  
        preferred.append("down")  
  
    # С вероятностью 0.7 выбираем одно из "желательных" действий,  
    # если оно существует; иначе действуем случайно.  
    if preferred and random.random() < 0.7:  
        return random.choice(preferred)  
    return random.choice(actions)
```

```
def td0_value_estimation(  
    env,  
    episodes=500,  
    alpha=0.1,  
    gamma=0.95,  
    max_steps_per_episode=100  
):  
    # Таблица оценок V(s).  
    # Для простоты и наглядности храним значения в словаре.  
    V = {  
        (x, y): 0.0  
        for y in range(env.height)  
        for x in range(env.width)  
    }  
  
    history = []  
  
    for episode in range(episodes):  
        state = env.reset()  
        total_reward = 0.0  
  
        for step in range(max_steps_per_episode):  
            action = policy(state, env)  
            next_state, reward, done = env.step(state, action)
```

```

# TD-ошибка:
#  $\delta = r + \gamma V(s') - V(s)$ 
# Для терминального состояния будущая ценность не учитывается.
target = reward
if not done:
    target += gamma * V[next_state]

td_error = target - V[state]

# Обновление оценки состояния
V[state] += alpha * td_error

total_reward += reward
state = next_state

if done:
    break

history.append(total_reward)

if (episode + 1) % 50 == 0:
    avg_reward = sum(history[-50:]) / 50
    print(f"Эпизод {episode + 1:3d} | средняя награда за последние 50 = {avg_reward:7.3f}")

return V, history

def print_value_table(V, env):
    print("\nОценка функции ценности V(s):")
    for y in range(env.height):
        row = []
        for x in range(env.width):
            state = (x, y)
            if state == env.goal:
                row.append(" GOAL ")
            else:
                row.append(f"{V[state]:6.2f}")
        print(" | ".join(row))

if __name__ == "__main__":
    random.seed(42)

    env = GridWorld(width=4, height=4, goal=(3, 3))
    V, history = td0_value_estimation(
        env,
        episodes=500,
        alpha=0.1,
        gamma=0.95,
        max_steps_per_episode=100
    )

```


)

```
print_value_table(V, env)
```

5.1. Что делает этот код

Класс GridWorld описывает среду: размеры поля, стартовое состояние, цель и правило перехода. Метод step реализует динамику среды и возвращает новое состояние, награду и признак завершения эпизода.

Функция policy задает фиксированную стратегию поведения. Она не оптимальна, но чаще подталкивает агента вниз и вправо, то есть приблизительно к цели. Благодаря этому мы быстрее получаем обучающие переходы с содержательными наградами.

Функция td0_value_estimation выполняет основной цикл обучения. На каждом шаге вычисляется целевое значение $target = r + \gamma V(s')$, после чего текущая оценка $V(s)$ сдвигается в сторону цели на долю, задаваемую коэффициентом обучения α .

В конце печатается таблица оценок $V(s)$. Чем больше значение, тем перспективнее состояние с точки зрения ожидаемой суммарной награды при следовании заданной политике.

5.2. Как интерпретировать результат

Состояния, расположенные ближе к цели, как правило, получают более высокие значения $V(s)$, потому что из них проще и быстрее достичь положительной награды.

Если в некоторых клетках оценки остаются низкими или отрицательными, это означает, что из них до цели идти долго, а по пути накапливается штраф за шаги. Иногда низкие значения также возникают из-за того, что фиксированная стратегия редко выбирает хорошие действия в этих областях.

Важный вывод: мы оценили не «абсолютную ценность клетки вообще», а ценность клетки при конкретной стратегии поведения. Если изменить стратегию, изменится и функция $V(s)$.

6. Типичные ошибки при постановке задач на оценку функции

- Смещение понятий «оценка функции» и «поиск оптимальной стратегии». Это разные этапы; сначала полезно отдельно понять оценку.
- Плохое описание состояния: слишком мало признаков или, наоборот, много лишней информации.
- Неверное масштабирование признаков при использовании линейных моделей и нейросетей.
- Слишком большой шаг обучения α , из-за которого оценки начинают колебаться.
- Недостаточное число эпизодов для получения устойчивых оценок.
- Неправильная обработка терминальных состояний — будущую ценность после конца эпизода добавлять нельзя.

7. Отдельный Python-скрипт с другой задачей: оценка $Q(s, a)$ методом SARSA

Ниже предлагается отдельная учебная задача, отличающаяся от предыдущего примера. Здесь агент не просто оценивает состояния, а учится оценивать пары «состояние–действие». Это позволяет сразу видеть, какое действие лучше в каждой точке.

Среда — коридор 6×4 с ямой-ловушкой. Агент стартует слева внизу и должен дойти до цели справа. За обычный шаг начисляется -1 , за попадание в ловушку -15 , за достижение цели $+20$. SARSA обучает $Q(s, a)$ по реально выполненной последовательности действий.

```
import random

# =====
# Отдельная задача:
# Оценка функции ценности действия  $Q(s, a)$  методом SARSA.
#
# Среда:
# - прямоугольное поле  $6 \times 4$ ;
# - есть цель и несколько "опасных" клеток;
# - агент стартует в левом нижнем углу;
# - за шаг начисляется  $-1$ , за ловушку  $-15$ , за цель  $+20$ .
#
# Смысл примера:
# 1. Мы оцениваем не  $V(s)$ , а  $Q(s, a)$ .
# 2. Это позволяет понять, какое действие лучше в каждом состоянии.
# 3. SARSA обновляет оценку по реально выбранному следующему действию:
#     $Q(s, a) \leftarrow Q(s, a) + \alpha * [r + \gamma * Q(s', a') - Q(s, a)]$ 
# =====

class CorridorEnv:
    def __init__(self, width=6, height=4):
        self.width = width
        self.height = height
        self.start = (0, height - 1)
        self.goal = (width - 1, height - 1)
        self.traps = {(2, height - 1), (3, height - 1), (4, height - 1)}

    def reset(self):
        return self.start

    def step(self, state, action):
        x, y = state

        if action == "up":
            y = max(0, y - 1)
        elif action == "down":
            y = min(self.height - 1, y + 1)
        elif action == "left":
            x = max(0, x - 1)
        elif action == "right":
            x = min(self.width - 1, x + 1)
```

```

        next_state = (x, y)

        if next_state in self.traps:
            reward = -15.0
            done = True
        elif next_state == self.goal:
            reward = 20.0
            done = True
        else:
            reward = -1.0
            done = False

        return next_state, reward, done

ACTIONS = ["up", "down", "left", "right"]

def epsilon_greedy(Q, state, epsilon):
    # epsilon-жадная стратегия:
    # иногда исследуем случайные действия,
    # а обычно берем действие с наибольшей Q-оценкой.
    if random.random() < epsilon:
        return random.choice(ACTIONS)

    values = [Q[(state, a)] for a in ACTIONS]
    best_value = max(values)

    # Если несколько действий одинаково хороши,
    # выбираем одно из них случайно.
    best_actions = [a for a in ACTIONS if Q[(state, a)] == best_value]
    return random.choice(best_actions)

def sarsa_train(
    env,
    episodes=600,
    alpha=0.1,
    gamma=0.95,
    epsilon=0.2,
    max_steps_per_episode=100
):
    # Таблица Q-оценок.
    Q = {}
    for y in range(env.height):
        for x in range(env.width):
            for action in ACTIONS:
                Q[((x, y), action)] = 0.0

    rewards_history = []

```

```

for episode in range(epochs):
    state = env.reset()
    action = epsilon_greedy(Q, state, epsilon)
    episode_reward = 0.0

    for step in range(max_steps_per_episode):
        next_state, reward, done = env.step(state, action)
        episode_reward += reward

        if done:
            # Если эпизод завершился, будущая Q-ценность равна 0.
            target = reward
            Q[(state, action)] += alpha * (target - Q[(state, action)])
            break
        else:
            next_action = epsilon_greedy(Q, next_state, epsilon)
            target = reward + gamma * Q[(next_state, next_action)]
            Q[(state, action)] += alpha * (target - Q[(state, action)])

            state = next_state
            action = next_action

    rewards_history.append(episode_reward)

    # Постепенно уменьшаем исследование.
    epsilon = max(0.03, epsilon * 0.995)

    if (episode + 1) % 50 == 0:
        avg_reward = sum(rewards_history[-50:]) / 50
        print(f"Эпизод {episode + 1:3d} | средняя награда за последние 50 = {avg_reward:7.3f}")

    return Q, rewards_history

def print_policy(Q, env):
    print("\nПриближенная стратегия по Q(s, a):")
    for y in range(env.height):
        row = []
        for x in range(env.width):
            state = (x, y)

            if state == env.goal:
                row.append(" GOAL ")
            elif state in env.traps:
                row.append(" TRAP ")
            else:
                best_action = max(ACTIONS, key=lambda a: Q[(state, a)])
                row.append(best_action.center(6))

```

```

        print(" | ".join(row))

if __name__ == "__main__":
    random.seed(7)

    env = CorridorEnv(width=6, height=4)
    Q, rewards = sarsa_train(
        env,
        episodes=600,
        alpha=0.1,
        gamma=0.95,
        epsilon=0.2,
        max_steps_per_episode=100
    )

    print_policy(Q, env)

```

8. Двадцать заданий

1. Измените размер GridWorld на 5×5 и исследуйте, как меняется таблица оценок $V(s)$. Объясните, почему части поля начинают иметь более низкие значения.
2. Замените фиксированную стратегию на полностью случайную и сравните полученные оценки состояния с исходным вариантом.
3. Измените штраф за шаг с -0.2 на -1.0 . Проанализируйте, как это влияет на $V(s)$ и на скорость обучения.
4. Сделайте две терминальные клетки: одну с наградой $+10$, другую с наградой $+3$. Объясните, как должна измениться функция ценности.
5. Добавьте препятствия, через которые нельзя проходить. Проверьте, как изменится распределение значений $V(s)$ по полю.
6. Постройте визуализацию значений $V(s)$ в виде тепловой карты с помощью matplotlib.
7. Реализуйте оценку $V(s)$ методом Монте-Карло и сравните результаты с TD(0) на одинаковой среде.
8. Сравните TD(0) при $\alpha = 0.05$, $\alpha = 0.1$ и $\alpha = 0.5$. Сделайте вывод о влиянии скорости обучения на устойчивость оценок.
9. Проведите серию запусков при разных γ : 0.7, 0.9, 0.99. Объясните, как меняется «дальновидность» агента.
10. Измените политику так, чтобы агент чаще ошибался и шел в направлении, противоположном цели. Покажите, как ухудшаются оценки.
11. Реализуйте хранение статистики средних наград по эпизодам и постройте график обучения.
12. Для SARSA-примера измените расположение ловушек так, чтобы безопасный путь был длиннее, а короткий путь — рискованнее. Сравните поведение агента.
13. Добавьте в SARSA уменьшение α по мере роста номера эпизода и исследуйте, становится ли обучение стабильнее.
14. Реализуйте Q-learning для той же коридорной среды и сравните итоговую стратегию с SARSA.

15. Сделайте непрерывный признак состояния, например «расстояние до цели», и реализуйте линейную аппроксимацию $V(s)$ на основе нескольких признаков.
16. Предложите набор признаков состояния для задачи управления запасами на складе и объясните, какие из них должны входить в функцию ценности.
17. Разработайте постановку RL-задачи для банкомата или очереди обслуживания, в которой ценность состояния зависит от загрузки системы и времени ожидания.
18. Исследуйте, как изменится обучение, если терминальное состояние будет не одно, а несколько, и часть из них будет штрафными.
19. Подготовьте эксперимент по сравнению табличной оценки и нейросетевой аппроксимации на одной и той же задаче. Сформулируйте план эксперимента и метрики.
20. Опишите предметную область из финансов, логистики или производства, где без оценки функции ценности невозможно принять хорошее решение. Предложите признаки состояния и тип алгоритма.

9. Контрольные вопросы и рекомендации к отчету

- Чем различаются $V(s)$, $Q(s, a)$ и $A(s, a)$?
- Почему в задачах с большим числом состояний нельзя ограничиться таблицей?
- Что означает свойство Маркова и почему оно важно для оценки функции?
- Чем Монте-Карло отличается от TD-обучения?
- В чем разница между SARSA и Q-learning?
- Почему качество признаков состояния критически влияет на качество оценки функции?
- Какие плюсы и риски дает использование нейронных сетей для оценки ценности?
- Почему терминальные состояния нужно обрабатывать отдельно?

В отчете по лабораторной работе рекомендуется описать постановку задачи, структуру состояния, функцию награды, выбранный алгоритм, гиперпараметры, результаты нескольких запусков, интерпретацию таблицы или графика оценок, а также выводы о влиянии параметров и структуры среды на качество обучения.